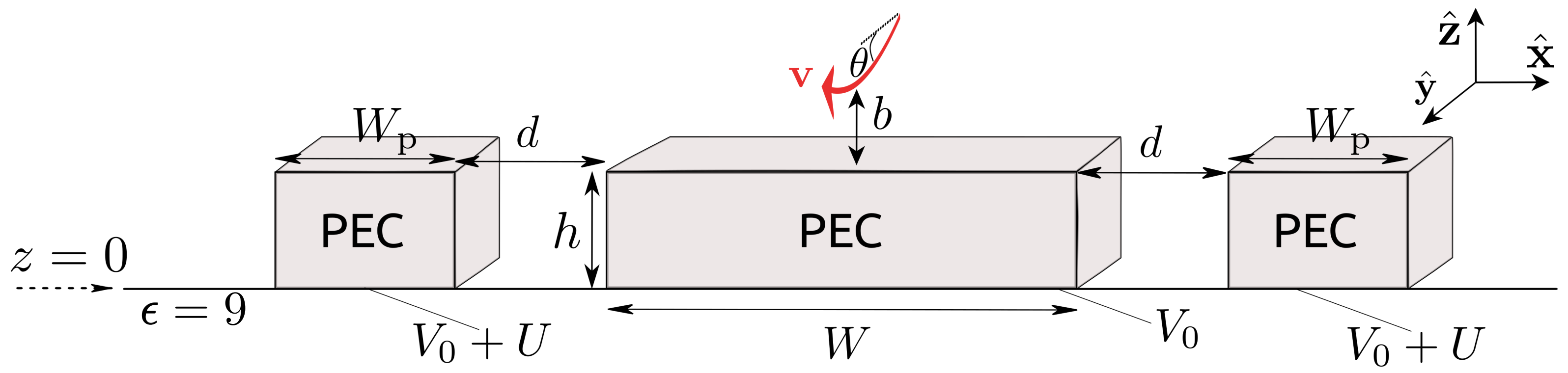
Electron-coupling to waveguide: code of the EELS

$$\Gamma_{\mathrm{EELS}}(\omega) = 2\int_{\underbrace{\boxed{b_{\min}+h}}_{z_{\min}}}^{\infty} \frac{v\,\mathrm{d}z}{\sqrt{\left|c^2\beta^2\sin^2\theta + \frac{2eV(x,z)}{m_{\mathrm{e}}\gamma_{\mathrm{e}}}\right|}}\;\underbrace{\frac{\mathrm{d}\Gamma_{\mathrm{EELS}}(\omega,x,z)}{\mathrm{d}y}}_{\text{step 1}}$$

step 3b

integration in step 4



1) run_EELS_rect_wg_along_z : obtain dEELS(z)/dy [1/eV*nm] for many energies

2) split_positive_eels.sh : split EELS in 1 file per 1 energy and retain only EELS >0

3) run_potential_and_find_bmin.sh :

a) Run e.out to create normalized potential data
g++ -o e.out e.cpp
files e.cpp and e.h

solutions of this Eq:

b) Run Python script to obtain bmin(V0,theta)
find_bmin_vs_V0_theta.py

$$0 = \frac{V(z_{\min})}{U}\frac{U}{V_0} + \frac{m_{\mathrm{e}}c^2\gamma_{\mathrm{e}}}{2e}\frac{\beta^2\sin^2\theta}{V_0} \longrightarrow \theta,\ V_0 \text{ for a fixed } z_{\min}$$

4) EELS_integrated_over_z.h and EELS_integrated_over_z.cpp: save "energy | Pintegrated [1/eV]" for each solution V0, theta
g++ -o EELS_integrated_over_z.out EELS_integrated_over_z.cpp

5) calculate_total_EELS_vs_theta.py: save "energy | EELS tot" for each solution V0, theta where EELS tot is the EELS integrated over z (data from step 4) and over the mode. Integration over the mode by fitting the data with a Lorentzian
fit_peaks.py